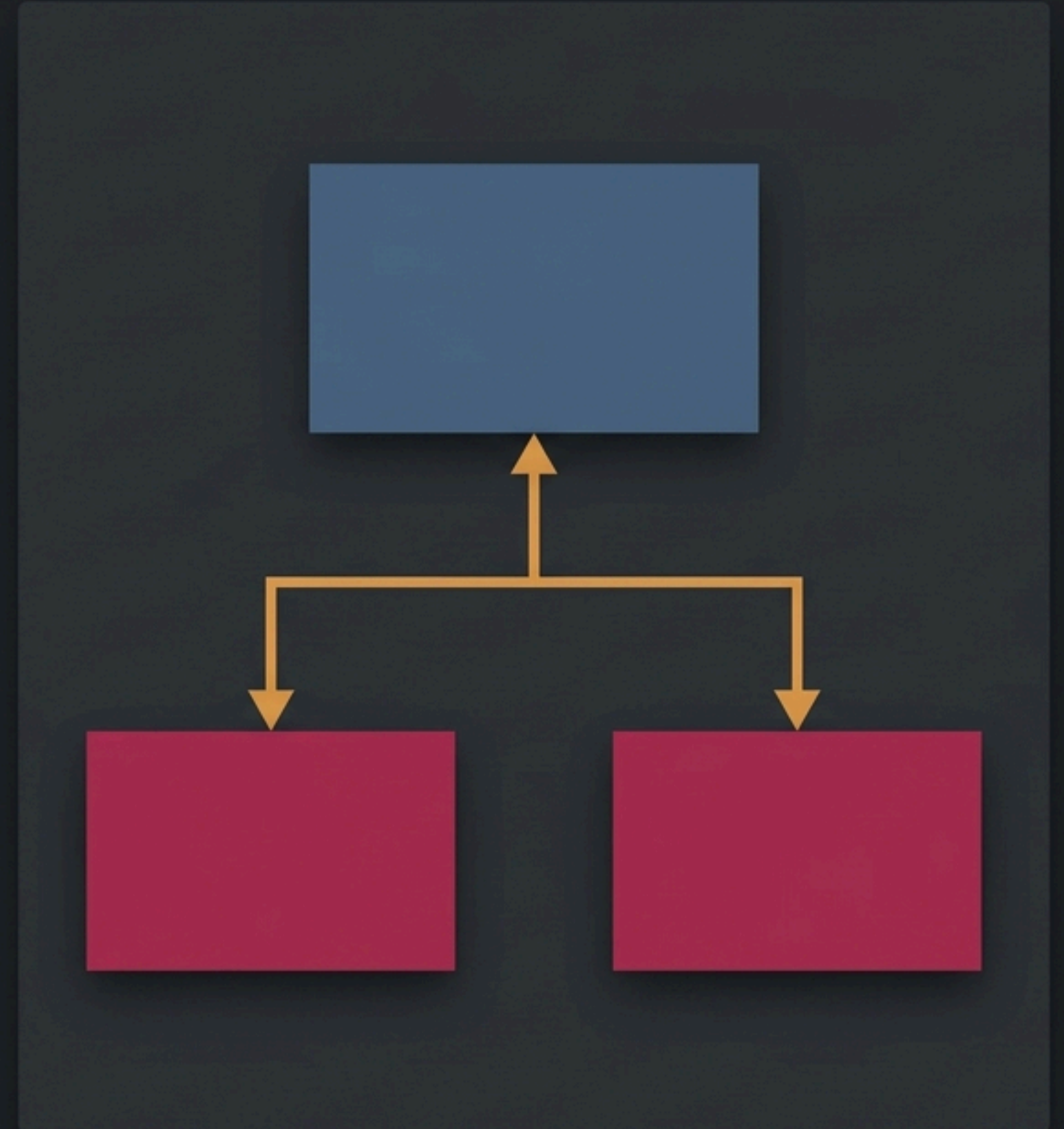


# El Plano Genético: Herencia en POO

Reutilización, Arquitectura  
y Sintaxis en Python.

---

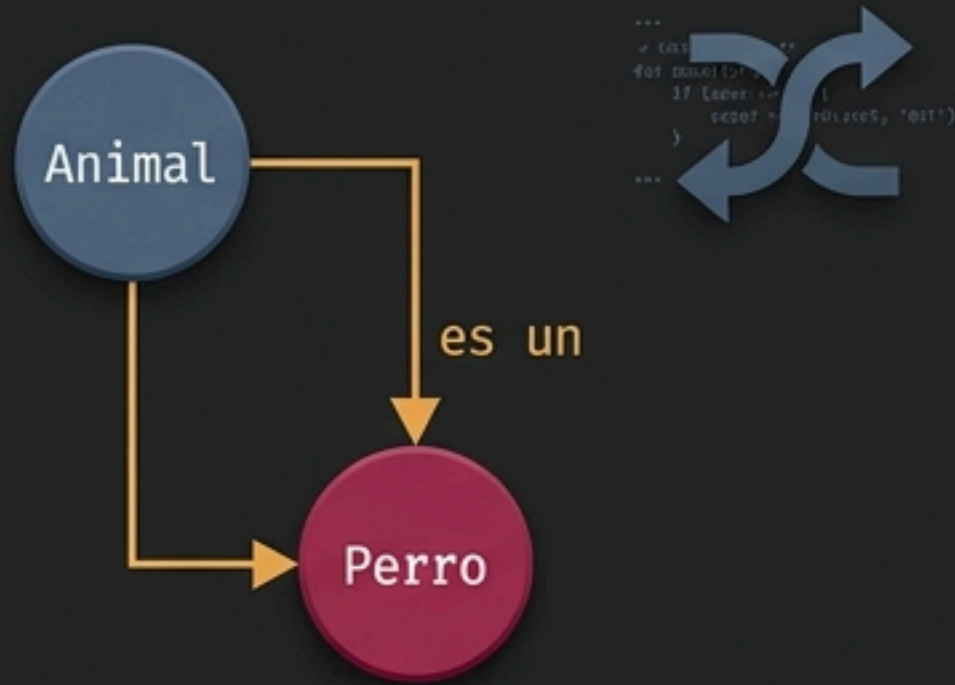
Una progresión interactiva:  
Aprender → Observar → Aplicar.





# Los 3 Puntos Clave de la Herencia

## Jerarquía y Reutilización



**Clase Padre (Base):** Clase existente con atributos y métodos generales compartidos.

**Clase Hija (Derivada):** Nueva clase que hereda características de la base.

**Relación:** Tipo 'es un'. Permite reutilizar código sin reescribirlo.

## Sintaxis y super()

```
class Hija(Padre):  
    super().__init__()
```

Diagrama de sintaxis de herencia. Un rectángulo gris contiene el código `class Hija(Padre):`. Una flecha curva naranja apunta desde el rectángulo hacia abajo y a la izquierda, donde se encuentra otro rectángulo gris con el código `super().__init__()`. Una flecha curva naranja también apunta desde el rectángulo superior hacia abajo y a la derecha, donde se encuentra un tercer rectángulo gris con el código `super().__init__()`.

**Sintaxis:** Se pasa el nombre del Padre entre paréntesis:  
`class Hija(Padre):`

**Función `super()`:** Llama al constructor de la clase padre desde la clase hija.

**Propósito:** Inicializar atributos heredados correctamente antes de agregar los propios.

## Sobrescritura de Métodos

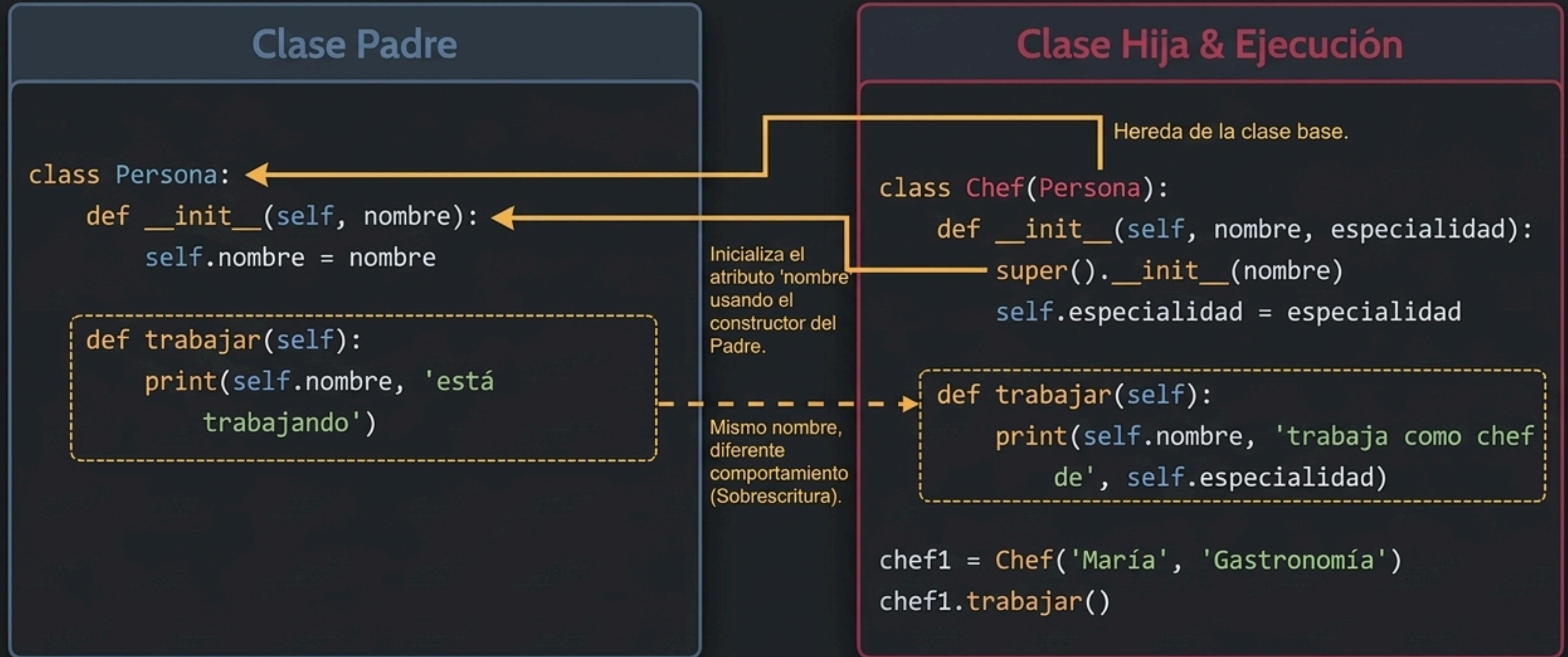


**¿Qué es?:** La clase hija redefine un método del padre utilizando exactamente el mismo nombre (Method Overriding).

**Propósito:** Modificar o adaptar el comportamiento de la clase padre para responder a necesidades específicas.



# Herencia en Python: Clase Padre, Clase Hija y Sobrescritura





## Ejercicio Práctico: Completando el ADN

```
class Animal:
    def __init__(self, nombre):
        self.nombre = nombre

    def hacer_sonido(self):
        print('El animal hace un sonido')

class Perro([ ]):
    def __init__(self, nombre, raza):
        [ ].__init__(nombre)
        self.raza = raza

    def [ ](self):
        print(f'{self.nombre} dice: ¡Guau!')

perro = Perro('Max', 'Labrador')
perro.[ ]()
```